

AWS DEVOPS INTERVIEW GUIDE

Scenario 1: Automatically Resize Images Uploaded to S3

Scenario

Your company has a website where users upload profile pictures to an S3 bucket. The application requires thumbnail images (200x200) to be created automatically whenever a new image is uploaded.

How would you implement this?

Solution

Architecture

```
graph TD
    User --> UploadImage[Upload Image]
    UploadImage --> S3Bucket[S3 Bucket]
    S3Bucket --> S3EventNotification[S3 Event Notification]
    S3EventNotification --> AWSLambda[AWS Lambda]
    AWSLambda --> ResizeImage[Resize Image]
    ResizeImage --> StoreThumbnail[Store Thumbnail in Another S3 Bucket]
```

Implementation Steps

1. Create two S3 buckets:
 - Original Images
 - Thumbnail Images
2. Configure an S3 Event Notification for `ObjectCreated`.
3. Create a Lambda function.
4. Lambda downloads the uploaded image.
5. Resize the image using an image processing library.
6. Upload the resized image to the thumbnail bucket.
7. Log execution details in CloudWatch Logs.

Benefits

- Fully automated

- Serverless
- Cost-effective
- Scales automatically

Services Used

- Lambda
- S3
- CloudWatch
- IAM

Scenario 2: Lambda Function Times Out

Scenario

A Lambda function that processes CSV files suddenly starts timing out after processing larger files.

How would you troubleshoot?

Solution

Step 1: Check CloudWatch Logs

Look for:

- Execution duration
- Memory usage
- Errors
- Timeout messages

Step 2: Verify Timeout

Current timeout:

3 Seconds

Increase if necessary:

60 Seconds

Maximum:

Step 3: Check Memory

Increasing memory also increases CPU allocation.

Example:

128 MB

↓

1024 MB

This often reduces execution time.

Step 4: Optimize Code

- Remove unnecessary loops
 - Process data in batches
 - Use efficient libraries
-

Step 5: Split Large Workloads

Instead of processing one huge file:

Large File

↓

SQS Queue

↓

Multiple Lambda Functions

Common Causes

- Large files
 - Slow database queries
 - External API delays
 - Low memory
 - Inefficient code
-

Scenario 3: Securely Access Database Credentials

Scenario

A Lambda function connects to an RDS database. Developers have hardcoded the username and password inside the code.

How would you secure it?

Solution

Use AWS Secrets Manager.

Architecture

Secrets Manager

↓

Lambda IAM Role

↓

Lambda Function

↓

Amazon RDS

Steps

1. Store credentials in Secrets Manager.
2. Assign permission to the Lambda execution role.
3. Retrieve the secret during execution.
4. Use the credentials to connect to the database.

Benefits

- No hardcoded passwords
 - IAM-based access
 - Automatic secret rotation
 - CloudTrail auditing
-

Scenario 4: Process Messages from an SQS Queue

Scenario

An application receives thousands of orders every minute. Orders should be processed asynchronously without overwhelming the backend.

How would you design this?

Solution

Architecture

Application

↓

Amazon SQS

↓

Lambda Trigger

↓

Process Order

↓

Database

Flow

1. Application sends messages to SQS.
2. Lambda polls the queue automatically.
3. Lambda processes messages in batches.
4. Successfully processed messages are removed.
5. Failed messages are retried or moved to a Dead Letter Queue (DLQ).

Benefits

- Decoupled architecture
 - Automatic scaling
 - Reliable processing
 - Retry capability
-

Scenario 5: API Backend Using Lambda

Scenario

Your company wants to build a REST API without managing servers.

How would you design it?

Solution

Architecture

Client

↓

API Gateway

↓

Lambda

↓

DynamoDB / RDS

Example Endpoints

GET /users

POST /users

PUT /users/{id}

DELETE /users/{id}

Benefits

- No server management
 - Automatic scaling
 - Pay per request
 - Easy integration with authentication and monitoring
-

Scenario 6: Lambda Cannot Access Resources in a Private VPC

Scenario

A Lambda function needs to connect to an RDS database in a private subnet, but connections are failing.

How would you troubleshoot?

Solution

Verify VPC Configuration

Lambda should be attached to:

- Correct VPC
 - Private subnets
 - Appropriate security groups
-

Verify Security Groups

Allow:

Lambda Security Group

↓

RDS Security Group

Port: 3306 / 5432

Verify Route Tables

If Lambda also needs internet access:

- NAT Gateway for private subnets

or

- VPC Endpoints for AWS services
-

Common Issues

- Incorrect security group rules
- Wrong subnet selection

- Missing NAT Gateway
 - Network ACL restrictions
 - Incorrect DNS settings
-

Scenario 7: Multiple Lambda Functions Need Common Code

Scenario

Ten Lambda functions use the same utility library for logging, authentication, and database connectivity.

How would you avoid duplicating code?

Solution

Use **Lambda Layers**.

Architecture

Lambda Layer

```
|— Logging Library
|— Utility Functions
|— Authentication Code
```

↓

Shared by Multiple Lambda Functions

Benefits

- Code reuse
 - Easier maintenance
 - Smaller deployment packages
 - Centralized updates
-

Scenario 8: Lambda Function Is Triggering Multiple Times

Scenario

A file uploaded to S3 results in duplicate Lambda executions, causing duplicate database records.

How would you prevent this?

Solution

Possible Causes

- Multiple S3 event notifications
 - Event retries
 - Duplicate uploads
-

Implement Idempotency

Before processing:

Check

↓

Already Processed?

↓

Yes → Ignore

No → Process

Store processed event IDs or object metadata in DynamoDB (or another persistent store).

Best Practices

- Design Lambda functions to be idempotent.
 - Use unique object keys or request IDs.
 - Monitor retry behavior.
-

Scenario 9: Schedule a Daily Report

Scenario

Every day at 6:00 AM, a report should be generated and emailed to management.

How would you implement this?

Solution

Architecture

Amazon EventBridge Scheduler

↓

Lambda

↓

Generate Report

↓

Amazon SES

Steps

1. Create a scheduled EventBridge rule (or EventBridge Scheduler).
2. Configure the schedule:

6:00 AM Daily

3. Lambda generates the report.
4. Send the report via Amazon SES.

Benefits

- Fully automated
 - No cron servers
 - Serverless scheduling
-

Scenario 10: Lambda Function Suddenly Becomes Slow

Scenario

Users report that an API backed by Lambda has become slow. The first request is significantly slower than subsequent requests.

How would you investigate?

Solution

Step 1: Check CloudWatch Metrics

Review:

- Duration
 - Errors
 - Throttles
 - Concurrent Executions
 - Init Duration (cold starts)
-

Step 2: Identify Cold Starts

Cold starts occur when AWS creates a new execution environment.

They are more noticeable when:

- Functions run infrequently
 - Deployment package is large
 - Function is attached to a VPC
 - Heavy initialization occurs during startup
-

Step 3: Optimize

- Reduce deployment package size.
 - Initialize expensive resources outside the handler where appropriate.
 - Allocate sufficient memory.
 - Reuse database connections when possible.
-

Step 4: Consider Provisioned Concurrency

For latency-sensitive applications:

Provisioned Concurrency

↓

Pre-initialized Lambda Environments

↓

Reduced Cold Starts

Step 5: Monitor Continuously

Use:

- CloudWatch Metrics
 - CloudWatch Logs
 - AWS X-Ray for distributed tracing
-

AWS Lambda Interview Cheat Sheet

Component	Purpose
Lambda Function	Executes code without managing servers
Event Source	Triggers Lambda (S3, SQS, API Gateway, EventBridge, DynamoDB Streams, etc.)
Execution Role	IAM role that grants Lambda access to AWS resources
Environment Variables	Store configuration values (non-sensitive values directly, sensitive values via Secrets Manager)
Lambda Layer	Share common libraries across functions
Dead Letter Queue (DLQ)	Stores failed asynchronous invocations for later analysis
Provisioned Concurrency	Reduces cold start latency
Reserved Concurrency	Limits or guarantees concurrency for a function
CloudWatch Logs	Captures execution logs
CloudWatch Metrics	Monitors invocations, duration, errors, throttles, concurrency
EventBridge	Schedules or routes events to Lambda
API Gateway	Exposes Lambda as REST or HTTP APIs
SQS	Enables asynchronous message processing
SNS	Invokes Lambda for pub/sub event processing
Secrets Manager	Securely stores credentials and API keys
AWS X-Ray	Provides tracing and performance analysis